МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №4

з дисципліни «Професійний Пайтон»

на тему: «Професійне ООП та система типів Python»

Виконав:
студент групи ФеП-32
Накладюк Володимир Тарасович
Варіант №15

Львів - 2026

1. Мета роботи

Сформувати практичні навички побудови професійної об’єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів, протоколів, dataclass, properties, dunder methods, принципів SOLID і сучасної системи type hints (Generic, TypeVar, TypedDict) з перевіркою коду статичним type checker mypy.

2. Індивідуальне завдання

Варіант 15. Система обліку результатів наукових експериментів. Domain objects: `Experiment`, `Measurement`, `Researcher`, `ExperimentResult`, `ExperimentSeries`. Реалізувати: додавання результату експерименту; пошук за дослідником; обчислення average/min/max результату; фільтрацію результатів за діапазоном; validation; repository; analysis strategy, exporter і reporting service.

3. Опис предметної області

Дослідник (researcher) проводить експерименти. Кожен експеримент вимірює один фізичний параметр в одній одиниці (наприклад, temperature у K) і накопичує серію результатів. Результат — це вимірювання (значення + параметр + одиниця) з датою і приміткою. Експерименти об’єднуються в серії (проєкти). Над результатами виконується аналіз різними стратегіями (середнє, мінімум, максимум, стандартне відхилення), а звіт експортується в різні формати.

Таблиця 4.1. Список domain entities та інших класів

Domain entity / об’єкт	Тип	Відповідальність
`Person` → `Researcher`	класи, наслідування	Ідентифікатор, ім’я, валідований e-mail, відділ.
`Measurement`	frozen dataclass (value object)	Значення, параметр, одиниця; порівняння і різниця сумісних вимірювань.
`ResultRange`	frozen dataclass (value object)	Діапазон [low, high] з оператором `in`.
`ExperimentResult`	dataclass	Один результат: ID, вимірювання, дата, примітка.
`Experiment`	клас-сутність	Назва, дослідник, параметр; володіє списком результатів.
`ExperimentSeries`	власна колекція	Набір експериментів із `len`, `in`, індексацією, ітерацією.
`InMemoryRepository[T]`	generic class	Зберігання будь-яких об’єктів з `id`.
`AnalysisStrategy`	ABC	Інтерфейс метрики; 4 конкретні стратегії.
`ReportExporter`, `ReportSink`	Protocol	Абстракції експорту і виводу звіту.
`ExperimentService`, `ReportingService`	service layer	Сценарії використання, отримують залежності через конструктор.

4. Хід виконання

4.1. Структура проєкту

Лістинг 4.1. Пакет experiment_registry.domain

```
domain/
├── __init__.py
├── dto.py
├── exporters.py
├── models.py
├── processing.py
├── protocols.py
├── repositories.py
├── services.py
├── strategies.py
└── value_objects.py
```

4.2. Class diagram

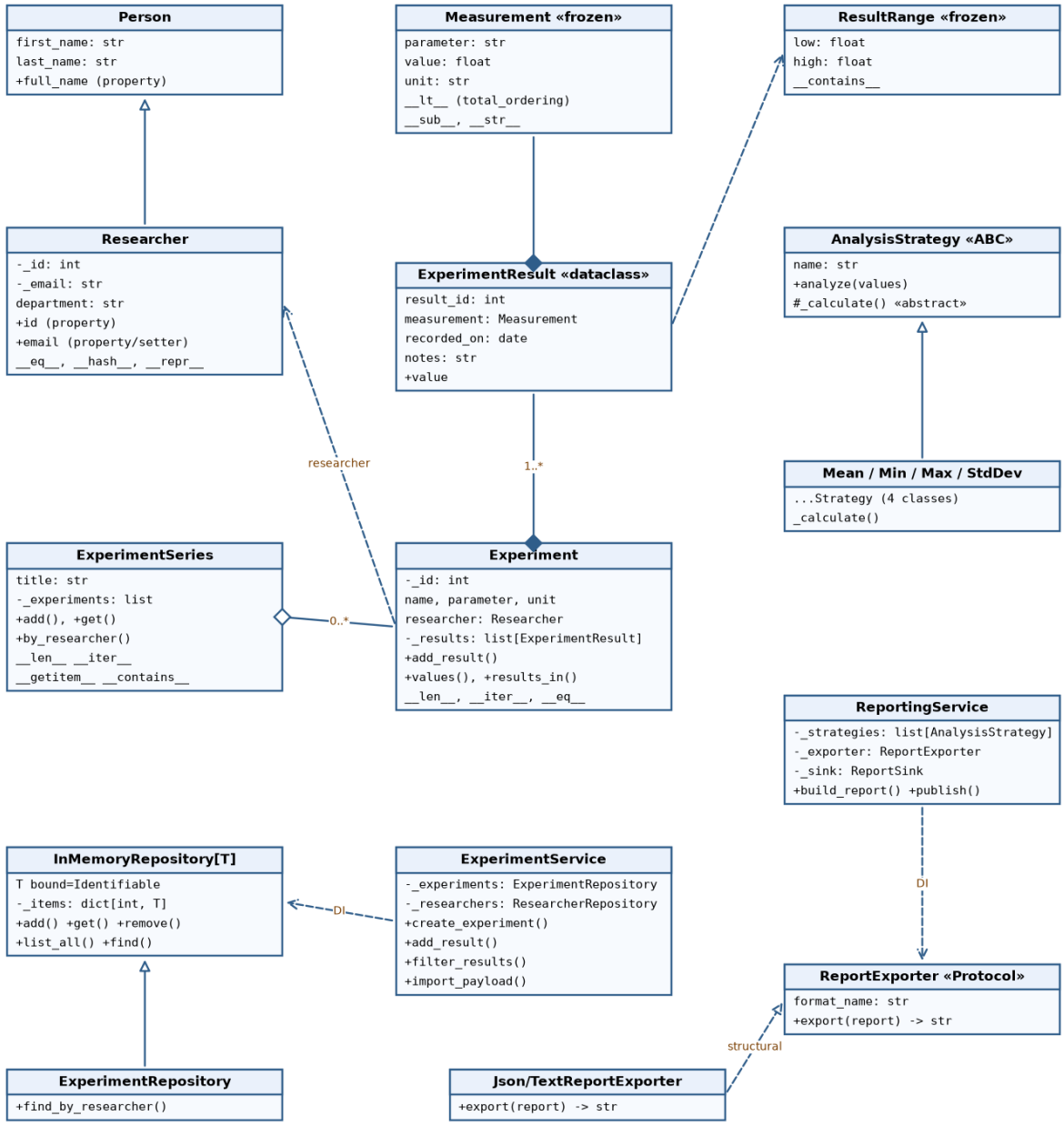


Рис. 4.1. Діаграма класів доменної моделі (суцільна стрілка з трикутником — наслідування, заповнений ромб — композиція, порожній ромб — агрегація, пунктир — залежність/реалізація протоколу)

4.3. Encapsulation і property

`Researcher` приховує стан у атрибутах `_id` і `_email`. Ідентифікатор доступний лише для читання через property `id`, а setter `email` нормалізує й перевіряє адресу, тому об’єкт не може опинитися в некоректному стані навіть після зміни e-mail. `Experiment` не віддає список результатів напряму: property `results` повертає незмінний tuple, а додати результат можна лише через `add_result()`, що перевіряє параметр, одиницю та унікальність ID.

Лістинг 4.2. Encapsulation і properties у класі Researcher

```
class Researcher(Person):
    """A scientist who performs experiments (inherits name handling from Person)."""

    def __init__(self, researcher_id: int, first_name: str, last_name: str,
                 email: str, department: str = "Physics") -> None:
        super().__init__(first_name, last_name)
        if researcher_id <= 0:
            raise ExperimentValidationError("Researcher ID must be positive.")
        self._id = researcher_id
        self.department = department
        self._email = ""
        self.email = email  # goes through the property setter

    @property
    def id(self) -> int:
        """Read-only identifier."""
        return self._id

    @property
    def email(self) -> str:
        return self._email

    @email.setter
    def email(self, value: str) -> None:
        value = value.strip().lower()
        if "@" not in value or value.startswith("@") or value.endswith("@"):
            raise ExperimentValidationError(f"Invalid e-mail: {value!r}.")
        self._email = value

    def __repr__(self) -> str:
        return f"Researcher(id={self._id}, name={self.full_name!r})"

    def __eq__(self, other: object) -> bool:
        return isinstance(other, Researcher) and other.id == self.id

    def __hash__(self) -> int:
        return hash(("Researcher", self._id))
```

4.4. Inheritance

Наслідування застосовано там, де виконується відношення «is-a» і принцип підстановки Лісков: `Researcher` є `Person` і перевикористовує перевірку імені через `super().__init__()`; конкретні стратегії є `AnalysisStrategy`; `ExperimentRepository` є спеціалізацією `InMemoryRepository[Experiment]` з додатковим запитом `find_by_researcher`.

Лістинг 4.3. Базовий клас Person

```
class Person:
    """Base class with a validated full name."""

    def __init__(self, first_name: str, last_name: str) -> None:
        if not first_name.strip() or not last_name.strip():
            raise ExperimentValidationError("First and last name are required.")
        self.first_name = first_name.strip()
        self.last_name = last_name.strip()

    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"
```

Лістинг 4.4. Наслідування від generic-класу

```
class ExperimentRepository(InMemoryRepository[Experiment]):
    """Repository with experiment-specific queries."""
```

```
    def find_by_researcher(self, researcher_name: str) -> list[Experiment]:
        wanted = researcher_name.strip().lower()
        return self.find(lambda item: item.researcher.full_name.lower() == wanted)
```

4.5. Composition

`Experiment` складається з об’єктів `ExperimentResult`, які, у свою чергу, містять `Measurement`. Результати створюються і живуть лише в межах експерименту (композиція), тоді як `ExperimentSeries` лише групує існуючі експерименти (агрегація). Сервіси також побудовані композицією: `ReportingService` має стратегії, exporter і sink.

Лістинг 4.5. Клас Experiment: композиція результатів

```
class Experiment:
    """Experiment entity; owns its results (composition)."""

    def __init__(self, experiment_id: int, name: str, researcher: Researcher,
                 parameter: str, unit: str) -> None:
        if experiment_id <= 0:
            raise ExperimentValidationError("Experiment ID must be positive.")
        if not name.strip():
            raise ExperimentValidationError("Experiment name must not be empty.")
        self._id = experiment_id
        self.name = name.strip()
        self.researcher = researcher
        self.parameter = parameter
        self.unit = unit
        self._results: list[ExperimentResult] = []

    @property
    def id(self) -> int:
        return self._id

    @property
    def results(self) -> tuple[ExperimentResult, ...]:
        """Read-only view; results can be added only through add_result()."""
        return tuple(self._results)

    def add_result(self, result: ExperimentResult) -> None:
        m = result.measurement
        if (m.parameter, m.unit) != (self.parameter, self.unit):
            raise IncompatibleMeasurementError(
                f"Experiment #{self._id} measures {self.parameter} [{self.unit}], "
                f"got {m.parameter} [{m.unit}]."
            )
        if any(item.result_id == result.result_id for item in self._results):
            raise ExperimentValidationError(f"Duplicate result ID {result.result_id}.")
        self._results.append(result)

    def values(self) -> list[float]:
        return [item.value for item in self._results]

    def results_in(self, interval: ResultRange) -> list[ExperimentResult]:
        return [item for item in self._results if item.measurement in interval]

    def __len__(self) -> int:
        return len(self._results)

    def __iter__(self) -> Iterator[ExperimentResult]:
        return iter(self._results)

    def __repr__(self) -> str:
        return (f"Experiment(id={self._id}, name={self.name!r}, "
                f"researcher={self.researcher.full_name!r}, results={len(self)})")

    def __eq__(self, other: object) -> bool:
        return isinstance(other, Experiment) and other.id == self.id

    def __hash__(self) -> int:
        return hash(("Experiment", self._id))
```

4.6. Dataclass і immutable value object

`ExperimentResult` — звичайний dataclass з перевіркою в `__post_init__`. `Measurement` і `ResultRange` — frozen dataclass: після створення поля змінити не можна (`FrozenInstanceError`), тому value objects безпечно передавати між об’єктами, використовувати як ключі dict і елементи set. `@total_ordering` генерує `<=`, `>`, `>=` з `__lt__` і `__eq__`.

Лістинг 4.6. Value objects

```
@total_ordering
@dataclass(frozen=True)
class Measurement:
    """A measured value of a physical parameter, e.g. 293.4 K of temperature.

    Frozen: equal measurements are interchangeable and can be dict keys or set items.
    """

    parameter: str
    value: float
    unit: str

    def __post_init__(self) -> None:
        if not self.parameter.strip():
            raise ExperimentValidationError("Measurement parameter must not be empty.")
        if not self.unit.strip():
            raise ExperimentValidationError("Measurement unit must not be empty.")
        if not math.isfinite(self.value):
            raise ExperimentValidationError(f"Measurement value must be finite, got
{self.value}.")

    def _check_compatible(self, other: "Measurement") -> None:
        if (self.parameter, self.unit) != (other.parameter, other.unit):
            raise IncompatibleMeasurementError(
                f"Cannot compare {self.parameter} [{self.unit}] with {other.parameter}
[{other.unit}]."
            )

    def __lt__(self, other: object) -> bool:
        if not isinstance(other, Measurement):
            return NotImplemented
        self._check_compatible(other)
        return self.value < other.value

    def __sub__(self, other: "Measurement") -> "Measurement":
        """Difference of two compatible measurements (arithmetic dunder)."""
        self._check_compatible(other)
        return Measurement(self.parameter, round(self.value - other.value, 6), self.unit)

    def __str__(self) -> str:
        return f"{self.value:g} {self.unit}"


@dataclass(frozen=True)
class ResultRange:
    """Closed interval [low, high] used to filter results."""

    low: float
    high: float

    def __post_init__(self) -> None:
        if self.low > self.high:
            raise ExperimentValidationError(f"Invalid range: {self.low} > {self.high}.")

    def __contains__(self, measurement: object) -> bool:
        if isinstance(measurement, Measurement):
            return self.low <= measurement.value <= self.high
        if isinstance(measurement, (int, float)):
            return self.low <= measurement <= self.high
        return False
```

Лістинг 4.7. Dataclass ExperimentResult

```
@dataclass
class ExperimentResult:
    """One recorded result of an experiment."""

    result_id: int
    measurement: Measurement
    recorded_on: date
    notes: str = ""

    def __post_init__(self) -> None:
        if self.result_id <= 0:
            raise ExperimentValidationError("Result ID must be positive.")
        if self.recorded_on > date.today():
            raise ExperimentValidationError("Result date cannot be in the future.")

    @property
    def value(self) -> float:
        return self.measurement.value
```

4.7. Abstract Base Class і Strategy

`AnalysisStrategy` успадковує `ABC` і має абстрактний метод `_calculate`; створити екземпляр базового класу неможливо. Публічний метод `analyze` реалізує шаблонний метод: спільна перевірка порожніх даних і округлення, а конкретне обчислення — у підкласах.

Лістинг 4.8. ABC та конкретні стратегії

```
class AnalysisStrategy(ABC):
    """Computes one metric from a sequence of result values."""

    name: str = "metric"

    def analyze(self, values: Sequence[float]) -> float:
        """Template method: common empty-data check, then the concrete calculation."""
        if not values:
            raise ValueError(f"{self.name}: no values to analyze")
        return round(self._calculate(values), 6)

    @abstractmethod
    def _calculate(self, values: Sequence[float]) -> float:
        """Concrete metric; implemented by subclasses."""


class MeanStrategy(AnalysisStrategy):
    name = "average"

    def _calculate(self, values: Sequence[float]) -> float:
        return sum(values) / len(values)


class StandardDeviationStrategy(AnalysisStrategy):
    """Sample standard deviation (0 for a single value)."""

    name = "std_dev"

    def _calculate(self, values: Sequence[float]) -> float:
        if len(values) < 2:
            return 0.0
        mean = sum(values) / len(values)
        return math.sqrt(sum((v - mean) ** 2 for v in values) / (len(values) - 1))
```

4.8. Protocol

Protocol описує структурний інтерфейс: клас відповідає протоколу, якщо має потрібні методи й атрибути, без явного наслідування. `JsonReportExporter` і `TextReportExporter` не успадковують `ReportExporter`, але mypy приймає їх там, де очікується протокол. `Identifiable` позначено `@runtime_checkable`, тому для нього працює `isinstance`.

Лістинг 4.9. Модуль protocols.py

```
"""Structural interfaces (Protocols) used for dependency inversion."""

from typing import Protocol, runtime_checkable

from experiment_registry.domain.dto import AnalysisReport


@runtime_checkable
class Identifiable(Protocol):
    """Anything with an integer `id` can be stored in the generic repository."""

    @property
    def id(self) -> int: ...


class ReportExporter(Protocol):
    """Serializes an analysis report into text (JSON, plain text, ...)."""

    format_name: str

    def export(self, report: AnalysisReport) -> str: ...


class ReportSink(Protocol):
    """Destination of exported reports (console, file, network)."""

    def write(self, text: str) -> None: ...
```

Лістинг 4.10. Exporters, сумісні з протоколом

```
class JsonReportExporter:
    format_name = "json"

    def __init__(self, indent: int | None = 2) -> None:
        self.indent = indent

    def export(self, report: AnalysisReport) -> str:
        return json.dumps(report, indent=self.indent, ensure_ascii=False)


class TextReportExporter:
    format_name = "text"

    def export(self, report: AnalysisReport) -> str:
        metrics = ", ".join(f"{key}={value:g}" for key, value in
report["metrics"].items())
        return (f"#{report['experiment_id']} {report['experiment']}
({report['researcher']}), "
                f"{report['parameter']} [{report['unit']}], n={report['count']}:
{metrics}")
```

4.9. Dunder methods

Таблиця 4.2. Реалізовані dunder methods

Метод	Клас	Призначення
`__eq__`, `__hash__`	`Researcher`, `Experiment`	Рівність сутностей за ідентифікатором
`__repr__`	`Researcher`, `Experiment`, `ExperimentSeries`	Налагоджувальне представлення
`__lt__` + `total_ordering`	`Measurement`	Порівняння сумісних вимірювань
`__sub__`	`Measurement`	Різниця двох вимірювань (арифметичний dunder)
`__str__`	`Measurement`	Вивід «301.8 K»
`__contains__`	`ResultRange`, `ExperimentSeries`, repository	Оператор `in`
`__len__`, `__iter__`	`Experiment`, `ExperimentSeries`, repository	`len()` і цикл `for`
`__getitem__` (з `@overload`)	`ExperimentSeries`	Індексація та зрізи

Лістинг 4.11. Власна колекція ExperimentSeries

```
class ExperimentSeries:
    """Custom collection of experiments (e.g. one research project)."""

    def __init__(self, title: str, experiments: list[Experiment] | None = None) -> None:
        self.title = title
        self._experiments: list[Experiment] = list(experiments or [])

    def add(self, experiment: Experiment) -> None:
        if experiment in self:
            raise ExperimentValidationError(f"Experiment #{experiment.id} is already in the
series.")
        self._experiments.append(experiment)

    def get(self, experiment_id: int) -> Experiment:
        for experiment in self._experiments:
            if experiment.id == experiment_id:
                return experiment
        raise ExperimentNotFoundError(f"Experiment #{experiment_id} not found in {self.title!
r}.")

    def by_researcher(self, researcher: Researcher) -> "ExperimentSeries":
        return ExperimentSeries(
            f"{self.title} / {researcher.full_name}",
            [item for item in self._experiments if item.researcher == researcher],
        )

    def all_values(self, parameter: str) -> list[float]:
        return [v for item in self._experiments if item.parameter == parameter for v in
item.values()]

    def __len__(self) -> int:
        return len(self._experiments)

    def __iter__(self) -> Iterator[Experiment]:
        return iter(self._experiments)

    def __contains__(self, item: object) -> bool:
        return item in self._experiments

    @overload
    def __getitem__(self, index: int) -> Experiment: ...

    @overload
    def __getitem__(self, index: slice) -> list[Experiment]: ...

    def __getitem__(self, index: int | slice) -> Experiment | list[Experiment]:
        return self._experiments[index]

    def __repr__(self) -> str:
        return f"ExperimentSeries({self.title!r}, experiments={len(self)})"
```

4.10. Generic class, TypeVar і TypedDict

`InMemoryRepository` параметризовано `TypeVar T` з обмеженням `bound=Identifiable`: репозиторій приймає лише об'єкти з `id`, а `InMemoryRepository[Experiment]` повертає саме `Experiment`, тому mypy перевіряє типи на місці використання. TypedDict `ExperimentPayload` описує структуру зовнішнього словника (з необов'язковим полем `notes` через `NotRequired`); це не доменна сутність, а лише форма даних на межі системи, яку сервіс перетворює на доменні об'єкти з перевіркою.

Лістинг 4.12. Generic repository

```
class InMemoryRepository(Generic[T]):
    """Stores objects of one type by their `id` (dict index => O(1) access)."""

    def __init__(self) -> None:
        self._items: dict[int, T] = {}

    def add(self, item: T) -> None:
        if item.id in self._items:
            raise DuplicateExperimentError(f"Object with id={item.id} already exists.")
        self._items[item.id] = item

    def get(self, item_id: int) -> T:
        try:
            return self._items[item_id]
        except KeyError:
            raise ExperimentNotFoundError(f"Object with id={item_id} not found.") from None

    def remove(self, item_id: int) -> T:
        item = self.get(item_id)
        del self._items[item_id]
        return item

    def list_all(self) -> list[T]:
        return list(self._items.values())

    def find(self, predicate: Callable[[T], bool]) -> list[T]:
        return [item for item in self._items.values() if predicate(item)]

    def __len__(self) -> int:
        return len(self._items)

    def __iter__(self) -> Iterator[T]:
        return iter(self._items.values())

    def __contains__(self, item_id: object) -> bool:
        return item_id in self._items
```

Лістинг 4.13. TypedDict у dto.py

```
"""TypedDicts for external payloads and reports (plain JSON-like data, not entities)."""

from typing import NotRequired, TypedDict


class ExperimentPayload(TypedDict):
    """A record as it arrives from outside (CSV/JSON/API)."""

    experiment_id: int
    name: str
    researcher: str
    parameter: str
    result: float
    unit: str
    date: str
    notes: NotRequired[str]
```

```
class AnalysisReport(TypedDict):
    """Result of applying analysis strategies to one experiment."""

    experiment_id: int
    experiment: str
    researcher: str
    parameter: str
    unit: str
    count: int
    metrics: dict[str, float]
```

4.11. Application service і dependency injection

Сервісний шар не створює залежностей сам: репозиторії, стратегії, exporter і sink передаються в конструктор. Усі конкретні об’єкти збираються в одному місці — composition root `build_service()` у демонстраційному сценарії.

Лістинг 4.14. ExperimentService

```
class ExperimentService:
    """Use cases for experiments; repositories are injected (dependency injection)."""

    def __init__(self, experiments: ExperimentRepository, researchers: ResearcherRepository) ->
None:
        self._experiments = experiments
        self._researchers = researchers
        self._next_result_id = 1

    def register_researcher(self, researcher: Researcher) -> Researcher:
        self._researchers.add(researcher)
        return researcher

    def create_experiment(self, experiment_id: int, name: str, researcher_id: int,
                          parameter: str, unit: str) -> Experiment:
        researcher = self._researchers.get(researcher_id)
        experiment = Experiment(experiment_id, name, researcher, parameter, unit)
        self._experiments.add(experiment)
        return experiment

    def add_result(self, experiment_id: int, value: float, recorded_on: date,
                   notes: str = "") -> ExperimentResult:
        """Add a validated result; the measurement gets the experiment's parameter and unit."""
        experiment = self._experiments.get(experiment_id)
        result = ExperimentResult(
            result_id=self._next_result_id,
            measurement=Measurement(experiment.parameter, value, experiment.unit),
            recorded_on=recorded_on,
            notes=notes,
        )
        experiment.add_result(result)
        self._next_result_id += 1
        return result

    def find_by_researcher(self, full_name: str) -> list[Experiment]:
        return self._experiments.find_by_researcher(full_name)

    def filter_results(self, interval: ResultRange,
                       parameter: str | None = None) -> list[tuple[Experiment,
ExperimentResult]]:
        return [
            (experiment, result)
            for experiment in self._experiments
            if parameter is None or experiment.parameter == parameter
            for result in experiment.results_in(interval)
        ]

    def import_payload(self, payload: ExperimentPayload, email_domain: str = "lnu.edu.ua") ->
ExperimentResult:
        """Convert an external TypedDict payload into domain objects."""
        researcher = self._researchers.find_by_name(payload["researcher"])
        if researcher is None:
```

```
            first, _, last = payload["researcher"].partition(" ")
            researcher = self.register_researcher(Researcher(
                len(self._researchers) + 1, first, last or first,
                f"{first.lower()}.{(last or first).lower()}@{email_domain}",
            ))
        if payload["experiment_id"] not in self._experiments:
            self.create_experiment(payload["experiment_id"], payload["name"], researcher.id,
                                   payload["parameter"], payload["unit"])
        return self.add_result(payload["experiment_id"], payload["result"],
                               date.fromisoformat(payload["date"]), payload.get("notes", ""))
```

Лістинг 4.15. ReportingService

```
class ReportingService:
    """Builds analysis reports; strategies, exporter and sink are abstractions (DIP)."""

    def __init__(self, strategies: Sequence[AnalysisStrategy], exporter: ReportExporter,
                 sink: ReportSink) -> None:
        self._strategies = list(strategies)
        self._exporter = exporter
        self._sink = sink

    def build_report(self, experiment: Experiment) -> AnalysisReport:
        values = experiment.values()
        return AnalysisReport(
            experiment_id=experiment.id,
            experiment=experiment.name,
            researcher=experiment.researcher.full_name,
            parameter=experiment.parameter,
            unit=experiment.unit,
            count=len(values),
            metrics={s.name: s.analyze(values) for s in self._strategies},
        )

    def publish(self, experiments: Sequence[Experiment]) -> int:
        """Export reports for experiments that have results; return how many were written."""
        written = 0
        for experiment in experiments:
            if len(experiment) == 0:
                continue
            self._sink.write(self._exporter.export(self.build_report(experiment)))
            written += 1
        return written
```

4.12. Принципи SOLID

Таблиця 4.3. Застосування SOLID

Принцип	Як реалізовано
S — Single Responsibility	Модель зберігає й перевіряє стан, стратегія рахує одну метрику, exporter лише серіалізує, sink лише виводить, репозиторій лише зберігає.
O — Open/Closed	Нова метрика чи формат звіту додаються новим класом (`MedianStrategy`, `CsvReportExporter`) без зміни `ReportingService`.
L — Liskov Substitution	Будь-яка стратегія підставляється замість `AnalysisStrategy`; `Researcher` можна використовувати як `Person`.
I — Interface Segregation	Малі протоколи `ReportExporter`, `ReportSink`, `Identifiable`, а також `LineParser`, `FieldValidator`, `FieldFormatter`.
D — Dependency Inversion	Сервіси залежать від абстракцій (ABC, Protocol), конкретні реалізації передаються через конструктор.

4.13. Основний код програми та результат виконання

Лістинг 4.16. Composition root демонстраційного сценарію

```
def build_service() -> ExperimentService:
    """Composition root: concrete dependencies are created in one place."""
```

```
    service = ExperimentService(ExperimentRepository(), ResearcherRepository())
    service.register_researcher(Researcher(1, "Olena", "Kovalenko",
"o.kovalenko@lnu.edu.ua"))
    service.register_researcher(Researcher(2, "Taras", "Hnatiuk", "t.hnatiuk@lnu.edu.ua",
"Mechanics"))
    service.create_experiment(1, "Thermal conductivity of Al alloy", 1, "temperature",
"K")
    service.create_experiment(2, "Gas compression test", 2, "pressure", "kPa")
    service.create_experiment(3, "Heat exchanger efficiency", 1, "temperature", "K")
    for experiment_id, value, day in [
        (1, 293.4, "2026-02-03"), (1, 297.1, "2026-02-04"), (1, 301.8, "2026-02-10"),
        (2, 101.3, "2026-02-05"), (2, 99.8, "2026-02-06"), (2, 250.4, "2026-03-06"),
        (3, 348.9, "2026-03-02"), (3, 352.2, "2026-03-03"),
    ]:
        service.add_result(experiment_id, value, date.fromisoformat(day))
    return service
```

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab04$ experiment-registry domain
Experiments of Olena Kovalenko: [Experiment(id=1, name='Thermal conductivity of Al alloy',
researcher='Olena Kovalenko', results=3), Experiment(id=3, name='Heat exchanger efficiency',
researcher='Olena Kovalenko', results=2)]

Validation:
  ExperimentValidationError: Measurement value must be finite, got nan.
  ExperimentValidationError: Result date cannot be in the future.
  ExperimentValidationError: Invalid e-mail: 'wrong-email'.
  ExperimentValidationError: Invalid range: 400 > 100.

Value objects: 301.8 K - 293.4 K = 8.4 K; 301.8 K > 293.4 K: True; hashable: 1 unique
  IncompatibleMeasurementError: Cannot compare temperature [K] with pressure [kPa].

Results in [290, 310] K:
  #1 Thermal conductivity of Al alloy: 293.4 K (2026-02-03)
  #1 Thermal conductivity of Al alloy: 297.1 K (2026-02-04)
  #1 Thermal conductivity of Al alloy: 301.8 K (2026-02-10)

Reports (strategies + JSON exporter + console sink):
{"experiment_id": 1, "experiment": "Thermal conductivity of Al alloy", "researcher": "Olena Kovalenko",
"parameter": "temperature", "unit": "K", "count": 3, "metrics": {"average": 297.433333, "minimum":
293.4, "maximum": 301.8, "std_dev": 4.209909}}
Same service with another exporter injected (TextReportExporter):
#1 Thermal conductivity of Al alloy (Olena Kovalenko), temperature [K], n=3: average=297.433,
minimum=293.4, maximum=301.8, std_dev=4.20991
#3 Heat exchanger efficiency (Olena Kovalenko), temperature [K], n=2: average=350.55, minimum=348.9,
maximum=352.2, std_dev=2.33345

Series: ExperimentSeries('Thermal project', experiments=2); len=2; series[0]='Thermal conductivity of Al
 alloy'; first experiment in series: True
All temperature values: [293.4, 297.1, 301.8, 348.9, 352.2]
Experiment is Identifiable (runtime_checkable Protocol): True

Imported TypedDict payload -> [Experiment(id=4, name='Battery discharge cycle', researcher='Iryna
Melnyk', results=1)], result 3.71 V

Inheritance processor: #7 Olena Kovalenko: 348.9
Composition processor: #7 Olena Kovalenko: 348.9
```

Рис. 4.2. Виконання підкоманди domain

На знімку видно додавання результатів, пошук за дослідником, відхилення некоректних даних власними винятками, роботу value objects, фільтрацію за діапазоном, звіти зі стратегіями average/min/max/std_dev у форматах JSON і text (той самий сервіс з іншим exporter), роботу dunder methods колекції та перетворення TypedDict у доменні об’єкти.

5. Експериментальна частина

5.1. Статична перевірка типів mypy

mypy встановлено як dev-залежність (`pip install -e ".[dev]"`) і налаштовано в `pyproject.toml` у режимі `strict`: заборонено функції без анотацій, неявний `Any` тощо.

Лістинг 4.17. Налаштування mypy у pyproject.toml

```
[tool.mypy]
python_version = "3.11"
strict = true
files = ["src"]
```

Під час розробки mypy виявив реальну помилку: метод репозиторію називався `list`, і в анотації `list[T]` наступного методу ім’я `list` посилалося вже на цей метод, а не на вбудований тип:

Лістинг 4.18. Діагностика mypy до виправлення назви методу

```
src/experiment_registry/domain/repositories.py:38: error: Function
"experiment_registry.domain.
    repositories.InMemoryRepository.list" is not valid as a type  [valid-type]
src/experiment_registry/domain/repositories.py:62: error: Value of type "list?[T]" is not
    indexable  [index]
```

Метод перейменовано на `list_all()`, після чого перевірка проходить без помилок.

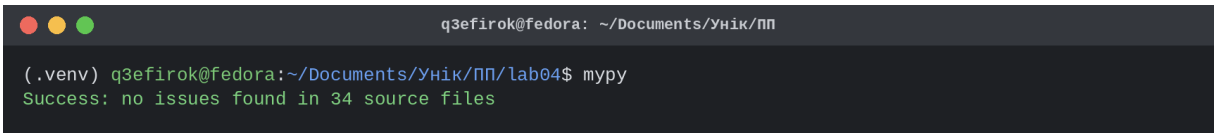


Рис. 4.3. Результат mypy для робочого коду

5.2. Навмисні помилки типів

Відповідно до завдання створено тимчасовий модуль із трьома помилками: рядок замість int, неправильний тип значення, що повертається, і додавання `Researcher` у `InMemoryRepository[Experiment]`.

Лістинг 4.19. Тимчасовий модуль type_errors_demo.py

```
"""Deliberate type errors for the type-checking experiment (temporary file)."""

from experiment_registry.domain.models import Experiment, Researcher
from experiment_registry.domain.repositories import InMemoryRepository

experiment_id: int = "1"


def total_results() -> int:
    return "100"


repository: InMemoryRepository[Experiment] = InMemoryRepository()
repository.add(Researcher(1, "Olena", "Kovalenko", "o.kovalenko@lnu.edu.ua"))
```

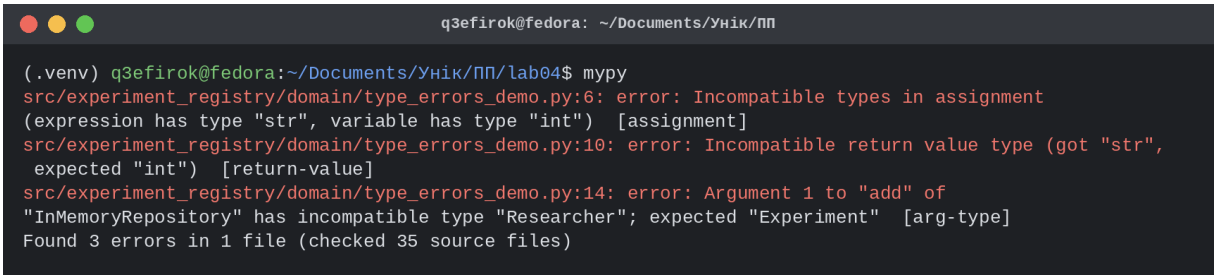


Рис. 4.4. Діагностика туру для навмисних помилок

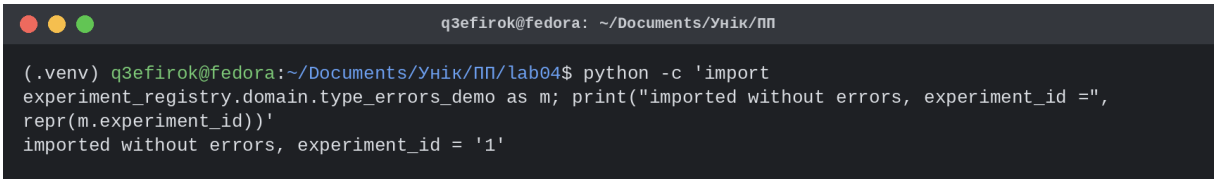


Рис. 4.5. Той самий модуль імпортується інтерпретатором без помилок

Python перевіряє типи лише під час виконання конкретних операцій, тому модуль імпортується без жодного попередження, а помилки проявилися б значно пізніше. Статичний аналіз знайшов усі три проблеми до запуску. Після видалення тимчасового модуля перевірка знову успішна.

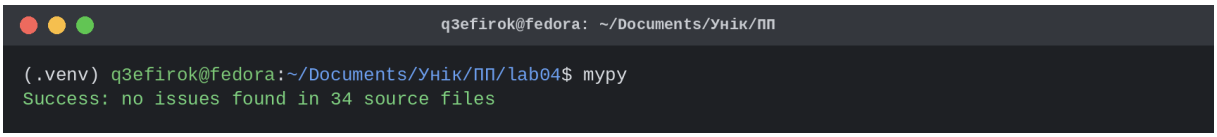


Рис. 4.6. Результат type checking після виправлення

5.3. Порівняння варіантів design: створення залежності vs dependency injection

Лістинг 4.20. Два варіанти побудови ReportingService

```
# Варіант А: сервіс сам створює конкретні залежності
class ReportingService:
    def __init__(self) -> None:
        self._strategies = [MeanStrategy(), MinimumStrategy(), MaximumStrategy()]
        self._exporter = JsonReportExporter()
        self._sink = ConsoleSink()


# Варіант В (реалізовано): залежності передаються ззовні
class ReportingService:
    def __init__(self, strategies: Sequence[AnalysisStrategy],
                 exporter: ReportExporter, sink: ReportSink) -> None:
        ...

ReportingService(default_strategies(), TextReportExporter(), MemorySink())
```

Таблиця 4.4. Порівняння варіантів design

Критерій	Варіант А	Варіант В (DI)
Тестування	Вивід завжди йде в консоль, підмінити exporter неможливо без patch	У тесті передається `MemorySink` або Mock і перевіряється результат
Заміна реалізації	Потрібно змінювати код сервісу	Достатньо передати інший об’єкт (JSON → text)
DIP	Порушено: сервіс залежить від конкретних класів	Виконано: залежність від ABC/Protocol

Критерій	Варіант A	Варіант B (DI)
Coupling	Високий	Низький: сервіс знає лише інтерфейс

Варіант B легше тестувати й розширювати, він відповідає принципу інверсії залежностей. Композиція з абстракціями знижує coupling, бо сервіс взаємодіє лише з контрактом (`export()`, `write()`, `analyze()`), а не з конкретним класом.

5.4. Inheritance vs composition

Одна й та сама поведінка (рядок CSV → перевірка діапазону → текстовий рядок) реалізована двома способами в модулі `processing.py`.

Лістинг 4.21. Варіант з наслідуванням

```
class BaseResultProcessor(ABC):
    def __init__(self, interval: ResultRange) -> None:
        self.interval = interval

    def process(self, line: str) -> str:
        fields = self.parse(line)
        self.validate(fields)
        return self.format(fields)

    @abstractmethod
    def parse(self, line: str) -> Fields: ...

    def validate(self, fields: Fields) -> None:
        if float(fields["result"]) not in self.interval:
            raise ExperimentValidationError(f"result {fields['result']} outside
{self.interval}")

    def format(self, fields: Fields) -> str:
        return f"#{fields['experiment_id']} {fields['researcher']}: {fields['result']}"


class CsvResultProcessor(BaseResultProcessor):
    def parse(self, line: str) -> Fields:
        return dict(zip(COLUMNS, line.strip().split(","), strict=True))
```

Лістинг 4.22. Варіант з композицією

```
class ResultProcessor:
    def __init__(self, parser: LineParser, validator: FieldValidator, formatter:
FieldFormatter) -> None:
        self.parser = parser
        self.validator = validator
        self.formatter = formatter

    def process(self, line: str) -> str:
        fields = self.parser.parse(line)
        self.validator.validate(fields)
        return self.formatter.format(fields)
```

Таблиця 4.5. Порівняння inheritance і composition

Критерій	Inheritance	Composition
Coupling	Підклас залежить від деталей базового класу	Компоненти пов’язані лише протоколами
Extensibility	Нова комбінація (інший формат + інший валідатор) — новий підклас	Нова комбінація збирається з наявних частин
Testability	Тестується лише весь клас	Кожен компонент тестується окремо, легко підставити fake
Повторне використання	Логіка прив’язана до ієрархії	`RangeValidator` можна використати деінде

5.5. Завдання підвищеної складності

- parameterized generic repository з bound TypeVar і спеціалізовані репозиторії;
- `runtime_checkable` Protocol `Identifiable` та interface segregation через кілька протоколів;
- власна колекція `ExperimentSeries` з `__getitem__`, `__iter__`, `__contains__`;
- арифметичний dunder `Measurement.__sub__` і `@total_ordering`;
- strategy pattern, repository abstraction, immutable value objects;
- ієрархія власних винятків (`ExperimentNotFoundError`, `IncompatibleMeasurementError`);
- type checker у вигляді однієї команди `mypy` з конфігурацією в pyproject.toml.

6. Висновок

У лабораторній роботі для проєкту experiment-registry побудовано типізовану об’єктну модель предметної області: п’ять доменних сутностей варіанта (`Researcher`, `Measurement`, `ExperimentResult`, `Experiment`, `ExperimentSeries`), value objects на frozen dataclass, наслідування там, де виконується «is-a», і композицію як основний спосіб побудови об’єктів. Інтерфейси описано через ABC і Protocol, сервіси отримують залежності через конструктор, що реалізує принципи SRP, OCP, LSP, ISP і DIP.

Generic-репозиторій з `TypeVar`, TypedDict для зовнішніх даних і повні type hints дозволили перевірити весь пакет mypy у режимі strict. Type checker знайшов як навмисні помилки, так і реальну (затінення вбудованого `list` назвою методу), яку інтерпретатор не помітив би до виконання. Експерименти показали, що dependency injection і композиція роблять код легшим для тестування й розширення, ніж жорстко створені залежності та глибоке наслідування.